

Actividad 3: Implementación de Attribute-Based Access Control (ABAC)

Tema: Control de acceso basado en atributos

Objetivo: Implementar ABAC en Flask



¿Qué es ABAC?

ABAC (Attribute-Based Access Control) usa atributos como **hora, ubicación, tipo de dispositivo** para otorgar acceso.

Configurar ABAC en Flask

Flask es un framework minimalista escrito en Python que permite **crear aplicaciones web rápidamente** y con un mínimo número de líneas de código. Está basado en la especificación WSGI de Werkzeug y el motor de templates Jinja2 y tiene una licencia BSD. <https://flask.palletsprojects.com/en/stable/>

Instalar Flask y Flask-Principal en un entorno virtual

1. Crear un entorno virtual en tu directorio actual:

```
python3 -m venv abac
```

2. Activar el entorno virtual:

```
source abac/bin/activate
```

```
(abac)-(root@kali)-[/home/kali]
# tree abac
abac
├── bin
│   ├── activate
│   ├── activate.csh
│   ├── activate.fish
│   └── Activate.ps1
├── pip
├── pip3
├── pip3.13
├── python → python3
├── python3 → /usr/bin/python3
├── python3.13 → python3
├── include
├── python3.13
├── lib
│   └── python3.13
│       └── site-packages
│           └── pip
│               ├── __init__.py
│               ├── internal
│               ├── build_env.py
│               ├── cache.py
│               └── cli
│                   ├── autocompletion.py
│                   └── base_command.py
```

3. Instalar los paquetes dentro del entorno virtual:

```
pip install flask flask-principal flask-login
```

4. Crear el archivo *abac_api.py*:

Este código incluye los criterios de acceso de: Rol del usuario (*user_role*), Horario de acceso (*access_time*), Dirección IP (*user_ip*) y Tipo de dispositivo (*user_agent*)

```
mousepad abac_api.py
```

```
from flask import Flask, request
from datetime import datetime
```

```
app = Flask(__name__)
```

```
# Definir direcciones IP permitidas y tipos de dispositivos permitidos
ALLOWED_IPS = {"127.0.0.1", "192.168.1.100"} # Puedes modificar según necesidad
ALLOWED_DEVICES = {"Mozilla", "Chrome", "Safari"} # Agrega más si es necesario
```

```
def check_access(user_role, access_time, user_ip, user_agent):
    """Función para verificar acceso basado en múltiples atributos"""
```

```
    # Acceso total para administradores
    if user_role == "admin":
        return True
```

```
    # Usuarios normales solo pueden acceder en horario laboral
    if user_role == "user" and (9 <= access_time.hour <= 18):
        # Verificar si la IP está permitida
        if user_ip in ALLOWED_IPS:
            # Verificar si el dispositivo está permitido
            for device in ALLOWED_DEVICES:
                if device in user_agent:
                    return True
```

```
    return False
```

```
@app.route('/dashboard')
```

```
def dashboard():
```

```
    user_role = request.args.get('role', 'guest') # Obtiene el rol del usuario
    access_time = datetime.now() # Obtiene la hora actual
    user_ip = request.remote_addr # Obtiene la IP del usuario
    user_agent = request.headers.get('User-Agent', '') # Obtiene el tipo de dispositivo
```

```
    if check_access(user_role, access_time, user_ip, user_agent):
        return "Acceso concedido"
```

```
    else:
        return "Acceso denegado", 403
```

```
if __name__ == '__main__':
    app.run(debug=True)
```

Probar la implementación de ABAC en Flask

1. Instalar dependencias

Antes de ejecutar la API, asegurarse de tener instaladas las bibliotecas necesarias dentro del entorno creado anteriormente:

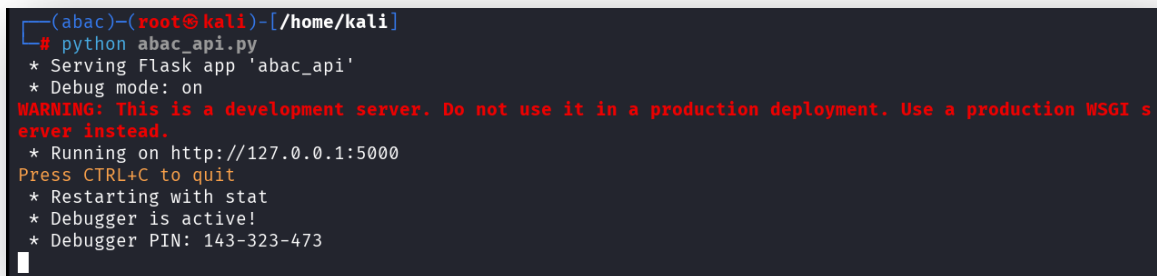
```
pip install flask flask-principal flask-login
```

2. Ejecutar la API

Ejecutar el script `rbac_api.py` en la terminal o entorno de desarrollo:

```
python abac_api.py
```

La API se ejecutará en `http://127.0.0.1:5000/`



Pruebas de Acceso

Para verificar el correcto funcionamiento de ABAC:

1. Prueba como "admin"

```
curl "http://127.0.0.1:5000/dashboard?role=admin"
```

Resultado esperado: *Acceso concedido*

2. Prueba como "user" dentro del horario laboral (ej. 10 AM)

```
curl "http://127.0.0.1:5000/dashboard?role=user"
```

Resultado esperado: *Acceso concedido (si la IP y el dispositivo son válidos)*

```
(abac)-(root@kali)-[/home/kali]
# curl -A "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:91.0) Gecko/20100101 Firefox/91.0" "http://127.0.0.1:5000/dashboard?role=user"
Acceso concedido
```

Resultado esperado: *Acceso denegado (si la IP o el dispositivo no son válidos)*

```
(abac)-(root@kali)-[/home/kali]
# curl "http://127.0.0.1:5000/dashboard?role=user"
Acceso denegado
```

```
(abac)-(root@kali)-[/home/kali]
# curl http://127.0.0.1:5000/dashboard?role=user --verbose
* Trying 127.0.0.1:5000 ...
* Connected to 127.0.0.1 (127.0.0.1) port 5000
* using HTTP/1.x
> GET /dashboard?role=user HTTP/1.1
> Host: 127.0.0.1:5000
> User-Agent: curl/8.12.1
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 403 FORBIDDEN
< Server: Werkzeug/3.1.3 Python/3.13.2
< Date: Tue, 04 Mar 2025 10:45:30 GMT
< Content-Type: text/html; charset=utf-8
< Content-Length: 15
< Connection: close
* shutting down connection #0
Acceso denegado
(abac)-(root@kali)-[/home/kali]
#
```

3. Prueba como "user" fuera del horario laboral (ej. 8 PM)

Resultado esperado: *Acceso denegado*

4. Prueba desde una IP no permitida

Si la IP no está en `ALLOWED_IPS`, el acceso debe ser denegado.

5. Prueba desde un dispositivo no permitido

Modifica `ALLOWED_DEVICES` para simular el rechazo de dispositivos.

Mejores Prácticas para ABAC

- No basar la seguridad en un solo atributo: Una buena práctica de seguridad es no depender de un solo atributo para autenticar usuarios o validar accesos. En su lugar, se deben combinar múltiples factores, como horario, IP, dispositivo y autenticación multi-factor (MFA), para mejorar la protección contra accesos no autorizados.
 - o *Riesgo de ataques*: Si solo se usa una contraseña, un atacante que la robe puede acceder sin restricciones.
 - o *Vulnerabilidad a ataques de fuerza bruta*: Sin controles adicionales, un atacante puede probar muchas combinaciones de credenciales.
 - o *Suplantación de identidad*: Si solo se usa la IP, un atacante con una VPN puede eludir la restricción.
 - o **Solución**: Combinar múltiples factores para una seguridad más robusta.
- Combinar ABAC con RBAC: Usar ABAC junto con Role-Based Access Control (RBAC) para una seguridad más fuerte.
 - o RBAC define qué roles existen y sus permisos básicos.
 - o ABAC refina el acceso según atributos contextuales.
 - o *Ejemplo*: Un usuario con rol "Editor" puede modificar contenido, pero solo si está dentro del horario laboral y conectado desde una IP autorizada.
- Registrar intentos de acceso es una práctica esencial de seguridad para detectar intentos de intrusión, ataques de fuerza bruta y accesos sospechosos en una aplicación o sistema. Esto permite auditoría, monitoreo y respuestas automáticas ante amenazas.
 - o *Detectar ataques de fuerza bruta*: Si un usuario intenta iniciar sesión muchas veces con credenciales incorrectas, puede ser un ataque.
 - o *Identificar accesos sospechosos*: Inicios de sesión desde ubicaciones o dispositivos desconocidos.
 - o *Cumplimiento de normativas*: ISO 27001, GDPR, PCI-DSS exigen auditorías de accesos.
 - o *Análisis forense*: En caso de un incidente, los registros ayudan a determinar qué ocurrió.
- Usar autenticación y tokens seguros: Para garantizar un acceso seguro a APIs y aplicaciones, se deben usar métodos de autenticación robustos como OAuth 2.0, JWT o API Keys. Además, combinar estos métodos con ABAC (Attribute-Based Access Control) permite aplicar restricciones dinámicas en función del contexto del usuario.